Capstone Project: Shopping Bot, Evaluation and Agent Ops

Start Assignment

Due Tuesday by 11:59pm **Points** 100 **Submitting** a file upload **File Types** pdf, zip, tgz, tar, docx **Attempts** 0 **Allowed Attempts** 1
Available Apr 12 at 10am - May 5 at 11:59pm

Weekly Outcomes Addressed

- Formulate a problem statement for an AI agent.
- Design and implement a functional AI agent.
- Evaluate agent performance against objectives.
- Present and justify design decisions.

Introduction

In this **Industry Capstone**, you will work as an **AI Engineer** or **Applied AI Consultant** inside a company. The organization wants to deploy an **AI agent** that can assist users in real workflows — handling ambiguity, reasoning across steps, using tools, learning from feedback, and operating safely in production.

Your responsibility is to **design**, **build**, **test**, **iterate**, and **justify** an AI agent as it would be done in an industry setting, not as a one-off prototype.

Problem Statement

You must design an AI agent that supports a realistic business workflow in one chosen industry scenario. The agent should demonstrate reliability, explainability, safety first behavior, and practical usefulness for real users, with evidence through artifacts and test logs.

Your final submission should make it clear:

- who the user is and what workflow the agent supports,
- what success looks like (criteria + metrics),
- what failures you anticipated and how you handled them,
- and why your architecture/design decisions are justified.

Framework Requirement

Choose **one** of the following tracks:

- **Track A — Framework-Based**: LangChain **OR** Crewai **OR** Flowise.
- **Track B — Framework-Free**: You must justify your architecture and show equivalent capabilities.

Note: If you choose **Flowise**, your build must be completed on Flowise (cloud or local). Flowise builds typically **cannot** be executed inside the **Vocareum** notebook environment, but you may still use Vocareum for course guidance and support resources.

Industry Scenarios (Choose ONE)

Select **one** scenario below and build your agent to meet the stated safety requirements.

Scenario 1: Business Operations — AI Operations Copilot (Decision Support Only)

Safety Requirements:

- Must refuse requests to modify data or trigger actions.
- Must explain uncertainty instead of guessing.
- Must provide escalation to a human analyst.
- Must not store sensitive business data in logs.

Scenario 2: Banking — AI Banking Support & Advisory Agent (Non-Transactional)

Safety Requirements:

- Must refuse money movement, approvals, or legal advice.
- Must not hallucinate customer data.
- Must escalate ambiguous or high-risk cases.
- Must not store PII in logs.

Scenario 3: Customer Support — AI Support Resolution Agent

Safety Requirements:

- Must refuse unsafe or policy-violating requests.
- Must not fabricate policies.
- Must escalate sensitive or unresolved cases.
- Must not store personal data in logs.

Task to be performed

You will build the agent following an industry workflow across the phases below. Your submission must include clear evidence (screenshots/logs/tables/artefacts) for the key capabilities.

Phase 1: Understand the Problem & Define Success

Coding: Not Required (optional)
Tools/Skills: Problem framing, user persona definition, workflow mapping, requirements writing, success criteria, edge-case thinking, evaluation planning

- Identify the primary user persona and daily workflow
- Document the exact problem to be solved
- Define inputs, outputs, constraints, and assumptions
- Write 3–5 example user questions
- Define success criteria
- List known failure cases and edge scenarios

Phase 2: Build a Basic Working Agent

Coding: Required
Tools/Skills: Python, CLI or notebook workflow, input/output handling, basic rules/templates, logging sample runs

- Create a Python-based agent
- Handle user input and generate responses
- Demonstrate baseline limitations

Tasks

- Create a Python-based agent that accepts user input
- Implement basic response generation (rules or templates)
- Demonstrate at least 2 limitations of the baseline agent
- Log sample interactions and responses
- Explain why this version is insufficient for real users

Phase 3: Make the Agent Smarter

Coding: Required
Tools/Skills: LLM integration (provider API), prompt engineering, prompt versioning, structured comparison of outputs, error/failure analysis

- Integrate an LLM
- Experiment with prompt strategies
- Show how behaviour changes with better prompts

Tasks

- Integrate an LLM into the agent workflow
- Design and test multiple prompt strategies
- Compare outputs across prompt variants
- Document improvements and new failure modes
- Select a default prompt strategy with justification

Phase 4: Add Knowledge & Retrieval

Coding: Required
Tools/Skills: Embeddings, semantic search, RAG concepts, text chunking, vector stores (e.g., Chroma/FAISS), retrieval-quality testing

- Implement embeddings and retrieval
- Enable document or data reference
- Show improvement over baseline

Tasks

- Prepare documents or datasets for embedding
- Implement semantic search using embeddings
- Connect retrieval results to agent responses
- Compare responses with and without retrieval
- Handle cases where relevant information is missing

Phase 5: Enable Tool Usage

Coding: Required
Tools/Skills: Tool/function calling, tool schemas, routing/selection logic, guardrails, error handling, loop prevention

- Allow the agent to choose and use tools
- Demonstrate correct vs incorrect tool usage
- Add safeguards

Tasks

- Define at least two tools the agent can use
- Implement tool calling logic
- Demonstrate correct tool selection
- Show at least one failed or incorrect tool call
- Add safeguards against misuse or loops

Phase 6: Planning, Memory & Context

Coding: Required
Tools/Skills: Planning/task decomposition, conversation state, short-term/long-term memory, memory reset/retention rules, multi-turn testing

- Introduce multi-step reasoning
- Add memory handling
- Improve multi-turn conversations

Tasks

- Implement multi-step reasoning or planning logic
- Add short-term or long-term memory
- Define memory retention and reset behaviour
- Demonstrate improved conversation quality

Phase 7: Adaptive Behaviour

Coding: Required
Tools/Skills: Feedback collection, storing feedback, behaviour adjustment logic, before/after comparisons, change explanation

- Introduce feedback signals
- Show behavioural change
- Explain the adaptation logic

Tasks

- Store feedback for future interactions
- Modify behaviour based on feedback
- Demonstrate before vs after behaviour
- Explain what changed and why

Phase 8: Deployment Readiness

Coding: Required
Tools/Skills: Packaging and reproducibility, environment management, basic deployment (local/cloud), logging/tracing, latency/error capture, graceful failure handling

- Package for deployment
- Add logging and tracing
- Handle runtime failures

Tasks

- Deploy locally or on the cloud
- Capture latency and error logs
- Demonstrate graceful failure handling
- Document deployment assumptions and limitations

Phase 9: Evaluation & Engineering Review

Coding: Required
Tools/Skills: Test harness design, evaluation prompts, quality/consistency metrics, failure analysis (root cause), safety/ethics review, improvement roadmap

- Measure response quality
- Analyze failures
- Review safety and ethics

Tasks

- Create evaluation prompts and test scenarios
- Measure quality and consistency metrics
- Perform root cause analysis
- Propose next-step improvements

Deliverables

Your final submission package must include:

1. **Working AI Agent**
2. **Problem Framing Document** (1–2 pages)
3. **Demo Script** (2–3 focused interactions)
4. **Evaluation Report**
5. **Engineering & Product Justification**

Evaluation Checklist

You are evaluated on **engineering judgment**, **reliability**, **explainability**, **safety-first design**, and **practical usefulness** in real workflows — not unnecessary complexity or academic novelty.

Minimum Bar (Required Evidence):

- Problem framing document
- Forced demo script
- Retrieval, tool usage, memory, and adaptation proof
- Evaluation report with root cause and fix
- Safety enforcement demonstration
- Framework usage or justified framework-free design

Required Method (Prompt Comparison Rule): You must demonstrate prompt evaluation using the **same test set**, **2–3 prompt variants**, and a **comparison table** (Prompt → Output → What Improved/Worsened).

How to Approach the Project

Build your capstone like an industry project: define success, ship a baseline, iterate with evidence, and document tradeoffs. Your artifacts should make your decisions easy to evaluate.

- Start with a clear persona + workflow and define success criteria early.
- Show the evolution: baseline → LLM → retrieval → tools → memory/planning → adaptation.
- Use logs/screenshots/tables as evidence — not just narrative claims.
- Demonstrate at least one failure case with root cause and a fix (before/after proof).
- Treat safety as a feature: refusals, uncertainty, escalation, and PII-safe logging.

Practice Guidelines

You may develop in your preferred environment (local or cloud) as long as your agent is reproducible and your submission includes complete evidence. If you are using an LLM provider (for example, the **OpenAI API**), ensure keys are handled securely and never exposed in shared files.

To Use Chat Support for This Assignment (Inside Vocareum):

- Open your practice notebook in **Vocareum**, then the left sidebar to open the **Assistant** panel.
- Ask for clarifications, troubleshooting help, or prompt setting support in the **Assistant** chat.
- **Important: Do not share API keys, credentials, or tokens in the Assistant panel**, and plan prompts accordingly.
- If you exhaust your allowance for a task, please contact the Emeritus support team.
- **API key access:** Your course-specific API key and usage notes are provided via the guide and Vocareum instructions. Follow those steps to locate your key when needed.
- You can find detailed instructions here: Vocareum_GenAI_API_Students_Guide.pdf

Security warning: Keep your API key private. Do not paste it in screenshots, chats, notebooks you share, or public repos. Prefer environment variables or secure key fields in tools; never hard-code keys in shared files. If you suspect exposure, rotate the key immediately per the guide.

To access Vocareum:

- Go to the *Getting Started* module on your Course homepage (just below Programme Orientation).
- Click the link labelled *Vocareum: Professional Certificate Programme in Agentic AI and Applications*.
- You can also access it from the Course menu on the left side of your screen.

Note: Your build environment depends on your chosen track and tooling. Ensure your submission includes clear artifacts that demonstrate retrieval, tool use, memory/planning, adaptation, evaluation, and safety enforcement.

Submission Guidelines

Submit **one zip file** containing the following:

- **Agent source** (code or Flowise export) with clear run instructions.
- **Problem framing document** (1–2 pages).
- **Demo script** (2–3 focused interactions) plus evidence (logs/screenshots).
- **Prompt comparison table** (same test set, 2–3 variants, insights).
- **Evaluation report** including at least one debugged failure case with root cause + fix + before/after proof.
- **Engineering & product justification** (design decisions, tradeoffs, safety approach, deployment assumptions).

Name your file as: **Capstone_Project_[Your Name].zip**

Note: This is a required graded activity and counts towards programme completion. Ensure your submission includes evidence for retrieval, tool usage, memory/planning, adaptation, evaluation, and safety enforcement. Your artifacts should make your design choices easy to evaluate.

Capstone Rubric

Criteria	Ratings			Pts
Problem Framing & Domain Understanding Domain understanding, Issue identification, Problem statement	10 to <0 pts **Excellent** Clear, realistic users/workflows/business context tied to the scenario; real world problems clearly justified with constraints/assumptions; explicit inputs/outputs/constraints/assumptions/success criteria/failure cases.	9 to <8.0 pts **Good** Context explained but partially generic; problems identified but weak justification; most problem statement elements present with minor gaps.	6 to <3.0 pts **Satisfactory** Basic context with limited realism; problems listed without justification; some elements missing.	2 to <0 pts **Needs Improvement** Context missing/incorrect; problems unclear or mismatched; problem statement largely incomplete.	10 pts
Python Foundations & Baseline Prototype (Code structure, Python fundamentals, Baseline functionality)	5 to <4.5 pts **Excellent** Clean, readable, logically separated components; correct use of functions/control flow/iterations with error handling; baseline runs reliably and clearly exposes limitations.	4.5 to <3.0 pts **Good** Mostly structured with minor coupling; fundamentals are mostly correct with minor issues; runs, but limitations are unclear.	3 to <1.5 pts **Satisfactory** Functional but messy structure; basic syntax only; partially functional baseline.	1.5 to <0 pts **Needs Improvement** Disorganized/unclear; frequent errors; baseline fails to run.	5 pts
LLM Integration & Structured Outputs LLM integration, Prompt design, Prompt comparison	15 to <13.5 pts **Excellent** Clean LLM integration into agent workflow with reusable/centralized prompts; support reasoning, safety, and task flow; structured prompt comparison table with insights/tradeoffs.	13.5 to <9.0 pts **Good** Integrated but loosely coupled; prompts work but lack structure; comparison shown but shallow.	9 to <4.5 pts **Satisfactory** Basic API call only; simple prompts; minimal comparison.	4.5 to <0 pts **Needs Improvement** Poor/broken integration; no meaningful prompt design; no real comparison.	15 pts
Embeddings & Semantic Retrieval (RAG) Embedding generation, Semantic search, Relevance analysis	10 to <9.0 pts **Excellent** Correct, efficient, and documented embedding pipeline; semantic retrieval clearly improves responses; relevance evaluated with examples.	9 to <6.0 pts **Good** Pipeline is correct but has limited documentation; retrieval works, but the impact is unclear; some relevance checking.	6 to <3.0 pts **Satisfactory** Basic embeddings; retrieval minimal; minimal relevance analysis.	3 to <0 pts **Needs Improvement** Embeddings/retrieval incorrect or missing; no relevance analysis.	10 pts
Tool Using Agent Implementation Tool design, Tool selection logic, Agent orchestration	15 to <13.5 pts **Excellent** Tools well-scoped; realistic, scenario-aligned; correct and reliable routing/selection logic; reliable multi-step execution with guardrails.	13.5 to <9.0 pts **Good** Tools useful but generic; mostly correct routing; execution mostly reliable.	9 to <4.5 pts **Satisfactory** Basic tools; hardcoded routing; partial reliability.	4.5 to <0 pts **Needs Improvement** Poorly designed tools; no real selection logic; fails frequently.	15 pts
Agent Architecture, Planning & Memory Architecture choice, Planning capability, Memory handling	15 to <13.5 pts **Excellent** Clear architecture with roles/flows/components; demonstrates task decomposition or multi-step planning; defined memory scope, retention, reset, and impact on quality.	13.5 to <9.0 pts **Good** Architecture mostly clear; some planning; memory used but weakly defined.	9 to <4.5 pts **Satisfactory** Basic structure; minimal planning; basic memory only.	4.5 to <0 pts **Needs Improvement** Architecture unclear; no planning; no memory.	15 pts
Adaptive Behaviour & Feedback Feedback signals, Behavior change, Limitations	5 to <4.5 pts **Excellent** Clear, realistic, explicit/implicit feedback design; observable system-level change with before/after logs; risks and limits clearly analysed.	4.5 to <3.0 pts **Good** Feedback defined but shallow; some change; some limits noted.	3 to <1.5 pts **Satisfactory** Partial deployment; minimal logs; minimal performance notes.	1.5 to <0 pts **Needs Improvement** No feedback signals; no behavior change; no limitations analysis.	5 pts
Deployment & Monitoring Deployment readiness, Monitoring & logging, Performance summary	10 to <9.0 pts **Excellent** Clean local/cloud packaging with assumptions; tracing/error handling/logs implemented; latency/quality summarized with evidence.	9 to <6.0 pts **Good** Deployment but undocumented; basic logs; performance mentioned.	6 to <3.0 pts **Satisfactory** Partial deployment; minimal logs; minimal performance notes.	3 to <0 pts **Needs Improvement** Not deployable; no monitoring/logging; no performance summary.	10 pts
Safety, Evaluation & Governance Evaluation metrics, Debugged failure case, Safety & ethics enforcement	15 to <13.5 pts **Excellent** Clear, repeatable quality/consistency metrics; debugged failure case with root cause + fix + before/after proof; demonstrates refusals, escalation, and PII-safe logging.	13.5 to <9.0 pts **Good** Metrics defined but shallow; root cause only; safety discussed + one demo.	9 to <4.5 pts **Satisfactory** Minimal metrics; failure noted only; safety discussed only.	4.5 to <0 pts **Needs Improvement** No metrics; no failure analysis/fix; safety/ethics ignored.	15 pts

Total Points: 100